HIJAB GRIP™ by HIJAB WORLD — Technical Project Report & Architectural Manual

Project Name: HIJAB GRIP by HIJAB WORLD **Version:** 1.0.0 (Public Showcase V1) **Date:** October 16, 2026 **Repository:** `hijab_world_v1` **Target Audience:** Junior Backend & Full-Stack Developers **Author / Generator:** Antigravity Gemini CLI **Last Commit Hash:** `5cac54adeaf358691e661899c79000da2e83415f`

> **NOTE: Source of Truth Disclaimer:** The real, executable code in this repository is the single source of truth. All architectural diagrams, schema models, API endpoints, and configuration parameters detailed within this report have been directly verified against active repository source files.

Table of Contents

1. [Executive Summary](#chapter-1-executive-summary) 2. [Quick Start for a New Developer](#chapter-2-quick-start-for-a-new-developer) 3. [Project Structure](#chapter-3-project-structure) 4. [Technology Stack](#chapter-4-technology-stack) 5. [System Architecture](#chapter-5-system-architecture) 6. [Frontend in Depth](#chapter-6-frontend-in-depth) 7. [Backend in Depth](#chapter-7-backend-in-depth) 8. [Database & Prisma ORM](#chapter-8-database--prisma-orm) 9. [API Reference & OpenAPI Specification](#chapter-9-api-reference--openapi-specification) 10. [Request & Data Flows](#chapter-10-request--data-flows) 11. [Docker & Containerized Infrastructure](#chapter-11-docker--containerized-infrastructure) 12. [Environment Variables](#chapter-12-environment-variables) 13. [Testing Strategy & Results](#chapter-13-testing-strategy--results) 14. [Security Architecture & Defense in Depth](#chapter-14-security-architecture--defense-in-depth) 15. [Performance, SEO & Accessibility](#chapter-15-performance-seo--accessibility) 16. [Scripts & Development Workflow](#chapter-16-scripts--development-workflow) 17. [Troubleshooting Guide (15 Realistic Scenarios)](#chapter-17-troubleshooting-guide) 18. [Planned or Removed Features & Known Limitations](#chapter-18-planned-or-removed-features) 19. [Technology Learning Guide & 1-Week Study Plan](#chapter-19-technology-learning-guide) 20. [Self-Check Quiz (30 Questions with Answer Key)](#chapter-20-self-check-quiz) 21. [Glossary (60 Core Technical Terms)](#chapter-21-glossary) 22. [Appendices](#chapter-22-appendices)

Chapter 1 Executive Summary

> **What you will learn here:** > - The business mission and architectural scope of Hijab World. > - What features are intentionally implemented versus deliberately excluded. > - The current operational status of the production showcase.

1.1 Purpose and Identity

Hijab World is a specialized modest fashion brand delivering the **HIJAB GRIP™**—an anti-slip, hypoallergenic headband constructed from 100% medical-grade silicone. It solves the friction, headaches, and slippage caused by traditional undercaps.

The website functions exclusively as a **Public Boutique Showcase**. It provides an interactive technical anatomy viewer, a curated foundation shade gallery, and a direct concierge inquiry channel.

1.2 Core Scope & Principles

1. **Public Showcase Only:** There are no customer user accounts, passwords, or login portals. 2. **Dedicated Social & Messaging Concierge:** All client contact is conducted via **WhatsApp** and **Instagram**.

- Official WhatsApp Number: `+91 8861720050` (Strict URL format: `https://wa.me/918861720050`).
- Official Instagram Profile: `https://www.instagram.com/hijab_worl`.
- **No Email:** No email addresses, contact forms, or `mailto:` links exist in the application.

3. **Pin-Free Modesty & Price-Free Gallery:** In accordance with luxury boutique principles, **no prices or currency symbols** are displayed anywhere. Instead, visitors tap **"Ask on WhatsApp"** to discuss tone matching and bundle delivery with a live stylist. 4. **Clean Codebase:** Administrative modules, staff logins, JWT authentication, and image upload pipelines have been removed to eliminate unnecessary attack surfaces.

Chapter 2 Quick Start for a New Developer

> **What you will learn here:** > - System prerequisites required on your workstation. > - How to bring up the database, backend, and frontend concurrently. > - Exact verification checks to confirm the stack is healthy.

2.1 System Prerequisites

- **Node.js:** v20.x or v24.x LTS (`node -v`)
- **npm:** v10.x or v11.x (`npm -v`)
- **Docker Desktop:** Required for local PostgreSQL containerization (`docker --version`)

2.2 First-Time Setup

Clone the repository and install dependencies in all subprojects:

```
# 1. Start the PostgreSQL 16 container via Docker Compose
docker compose up -d postgres

# 2. Setup the Backend Database Schema & Seed Data
cd backend
npm install
npx prisma db push
npm run seed
cd ..

# 3. Setup the Frontend
cd frontend
npm install
cd ..
```

2.3 Starting the Entire Stack Together

Launch both the NestJS API and Next.js frontend with one root command:

```
npm run dev:all
```

- **Next.js Web App:** `http://localhost:3000`
- **NestJS API Base:** `http://localhost:4000/api/v1`
- **API Health Check:** `http://localhost:4000/api/v1/health`

2.4 Automated Stack Verification

Run the integrated test suite that automatically confirms health, database connection, API contracts, security protections, and regression invariants:

```
npm run test:stack
```

Chapter 3 Project Structure

> **What you will learn here:** > - The monorepo-style layout separating root orchestration, backend, and frontend. > - The role of each directory and configuration file in active service.

```
hijab_world_v1/
├── docker-compose.yml       # Local PostgreSQL 16 Alpine container specification
├── package.json             # Root orchestration scripts (dev:all, test:stack)
├── scripts/
│   ├── start_stack.js       # Node.js runner to spawn backend & frontend in parallel
│   └── test_stack.js        # Health-check polling harness running tests against active stack
├── backend/                 # NestJS 12 REST API Microservice
│   ├── src/
│   │   ├── main.ts          # Application bootstrap, Helmet, strict CORS, port listener
│   │   ├── app.module.ts    # Root Nest module importing Prisma, Products, Settings, Throttler
│   │   ├── app.controller.ts# Root controller exposing GET /api/v1 and GET /api/v1/health
│   │   ├── app.service.ts   # Core service performing live PostgreSQL ping
│   │   ├── prisma/          # PrismaService database abstraction layer
│   │   ├── products/        # ProductsModule, ProductsController, ProductsService
│   │   └── settings/        # SettingsModule, SettingsController, SettingsService
│   ├── prisma/
│   │   ├── schema.prisma    # PostgreSQL database schema definitions
│   │   └── seed.ts          # Content seeding script for categories, products, and settings
│   ├── .env.example         # Template for environment variables (clean of secrets)
│   ├── package.json         # Backend dependencies and scripts
│   └── tsconfig.json        # TypeScript configuration for NestJS
├── frontend/                # Next.js 16.4 (Turbopack) Web Client
│   ├── src/
│   │   ├── app/
│   │   │   ├── layout.tsx   # Root HTML layout with viewport & font metadata
│   │   │   ├── page.tsx     # Single Page Application container with 4 interactive views
│   │   │   ├── globals.css  # Global Tailwind directives
│   │   │   └── api/settings/# Next.js server proxy route to /api/v1/settings
│   │   ├── config/site.ts   # Static client-side site metadata fallback
│   │   └── lib/links.ts     # Social link verification and formatting helpers
│   ├── public/
│   │   ├── app.js           # Client-side UI controller (navigation, hotspots, WhatsApp links)
│   │   ├── style.css        # Responsive styling and design tokens
│   │   ├── robots.txt       # Search engine crawler directives
│   │   ├── sitemap.xml      # XML sitemap
│   │   └── assets/images/   # High-resolution optimized product & anatomy imagery
│   ├── tests/               # Automated test suites
│   │   ├── api_contract.test.ts  # Node test suite verifying backend schema contracts
│   │   ├── phase3_frontend.test.ts# Multi-viewport responsive tests (1920px to 375px)
│   │   ├── phase4_security.test.ts# SQLi fuzzing, XSS, rate limiting, header checks
│   │   └── screenshots/     # Playwright captured visual regression baselines
│   ├── package.json         # Frontend dependencies and dev scripts
│   └── next.config.ts       # Next.js configuration with security headers
└── docs/                    # Architectural documents and full report
    ├── BLUEPRINT.md         # Original build blueprint
    └── report/              # Generated documentation, markdown, and compiled PDF
```

Chapter 4 Technology Stack

> **What you will learn here:** > - Every library, framework, and tool in active production and development use. > - Exact version numbers retrieved directly from package manifests. > - The specific rationale for each architectural dependency.

Technology / Library	Version	Category	Role in Hijab World	Why It Was Chosen
Node.js	`v24.19.0`	Runtime	JavaScript execution engine	Long Term Support runtime with native ESM and test runner.
Next.js	`16.4.0`	Frontend Framework	Client application architecture	Fast Turbopack bundler, serverless proxy routes, high CWV performance.
React	`19.3.0`	UI Library	Component rendering	Modern declarative UI component rendering.
Tailwind CSS	`^4.0.0`	Styling Engine	Utility-first CSS framework	Rapid styling with custom brand color tokens and zero runtime overhead.
NestJS	`^12.0.1`	Backend Framework	REST API Architecture	Structured modular architecture with dependency injection and TypeScript decorators.
Prisma ORM	`^6.19.3`	Database ORM	Schema and Query Abstraction	Type-safe SQL querying with automated migration generation.
PostgreSQL	`16-alpine`	Relational Database	Persistent Data Storage	ACID-compliant relational storage for products, categories, and settings.
Helmet	`^8.3.0`	Security Middleware	HTTP Response Security Headers	Automatic setting of standard security headers (CSP, X-Frame-Options, nosniff).
@nestjs/throttler	`^6.7.1`	Security / Rate Limit	Abuse prevention	Protects public endpoints from brute-force bursts (100 reqs/min).
Playwright	`^1.64.0`	Testing / Browser	E2E & Layout Verification	Headless browser engine for cross-device layout and link testing.
Vitest	`^4.1.2`	Testing Framework	Backend Unit Testing	Fast Vite-native unit test runner for NestJS services and controllers.
Docker	`24.x+`	Container Engine	Local Infrastructure	Reproducible PostgreSQL service container across development machines.

Chapter 5 System Architecture

> **What you will learn here:** > - How visitors interact with the Next.js frontend and NestJS backend. > - The network boundaries between localhost ports 3000, 4000, and 5434. > - Why frontend and backend operate as decoupled microservices.

5.1 Architecture Diagram

```
+-------------------------------------------------------------+
|                    Visitor Browser (Client)                 |
|      [Desktop 1440px / Mobile 375px]                        |
+------------------------------+------------------------------+
                               |
                               | HTTP (Port 3000)
                               v
+-------------------------------------------------------------+
|              Next.js 16.4 Frontend (Turbopack)              |
|  - Single Page Interactive Shell (Home, Anatomy, Packs, Order)  |
|  - Client-Side Controller (public/app.js)                   |
|  - Proxy Cache Route (src/app/api/settings/route.ts)        |
+------------------------------+------------------------------+
                               |
                               | HTTP REST / JSON (Port 4000)
                               | CORS: Allowed Origin http://localhost:3000
                               v
+-------------------------------------------------------------+
|                   NestJS 12 Backend API                     |
|  - Security Middleware: Helmet & Strict CORS                |
|  - Rate Limiting: ThrottlerGuard (100 req/60s)              |
|  - Endpoints: GET /health, GET /products, GET /settings     |
+------------------------------+------------------------------+
                               |
                               | Prisma Client Connection Pool (Port 5434)
                               v
+-------------------------------------------------------------+
|           PostgreSQL 16 Database (Docker Container)         |
|  - Container Name: hijab_world_postgres                     |
|  - Database: hijab_world_db (or hijab_world_test)           |
|  - Tables: categories, products, site_settings              |
+------------------------------+------------------------------+
                               |
                               | Outbound Navigation (External)
                               v
+-------------------------------------------------------------+
|                  External Concierge Services                |
|  - WhatsApp Business Link: https://wa.me/918861720050       |
|  - Instagram Profile: https://www.instagram.com/hijab_worl  |
+-------------------------------------------------------------+
```

5.2 Rationale for Decoupled Architecture

- **Fault Tolerance:** If the NestJS backend restarts or experiences network downtime, the Next.js frontend continues serving cached assets and activates graceful fallback WhatsApp concierge buttons.
- **Security Isolation:** Database credentials (`DATABASE_URL`) and backend internal modules reside strictly in the backend container environment, completely inaccessible to browser clients.

Chapter 6 Frontend in Depth

> **What you will learn here:** > - How the Single Page Application architecture manages 4 distinct views. > - The interactive anatomy hotspot inspection mechanics. > - The responsive viewport strategy and styling system.

6.1 View Routing & Navigation Shell

The application utilizes a Single Page Application (SPA) architecture in `src/app/page.tsx` supported by `public/app.js`. Hash-based routing (`#overview`, `#anatomy`, `#packs`, `#order`) synchronizes navigation state without full page reloads:

1. **Home Overview (`#overview`):**

- High-contrast hero section with value propositions.
- Patented lattice matrix badge and 3-card feature highlight.
- Comparison grid: Traditional fabric undercap vs. Silicone honeycomb matrix.
- 24/7 client care callout.

2. **Grip Anatomy & Tech (`#anatomy`):**

- Interactive medical-grade silicone specification diagram.
- 4 selectable ergonomic zones:
- *Zone 1 (Forehead Arc):* Eliminates sinus and temple pressure points.
- *Zone 2 (Honeycomb Air Matrix):* 96% airflow for scalp cooling.
- *Zone 3 (Bio-Grip Dual Surface):* Micro-suction outer texture holding sheer fabrics.
- *Zone 4 (Tension-Memory Elasticity):* Retains form over 1,000+ wears.
- Tap-to-inspect cards updating hotspot coordinates dynamically.

3. **Collection & Tones (`#packs`):**

- Signature Everyday Essentials Trio (3-Pack Bundle) with individual shade swatches.
- Responsive product variant grid:
- *Onyx Black:* Deep foundation for black modal and dark georgette.
- *Warm Caramel Nude:* Matches skin tones under sheer neutral silks.
- *Pure Ivory:* Formulated for bridal styling and light pastels.
- Verified boutique buyer testimonials.

4. **Customer Support & Order (`#order`):**

- Fast-track direct messaging cards for WhatsApp Concierge and Instagram Direct.
- Interactive FAQ accordion covering sizing, delicate fabric locking, and care instructions.

6.2 Visual Layout & Responsive Design Evidence

Desktop Showcase (1440px Viewport)

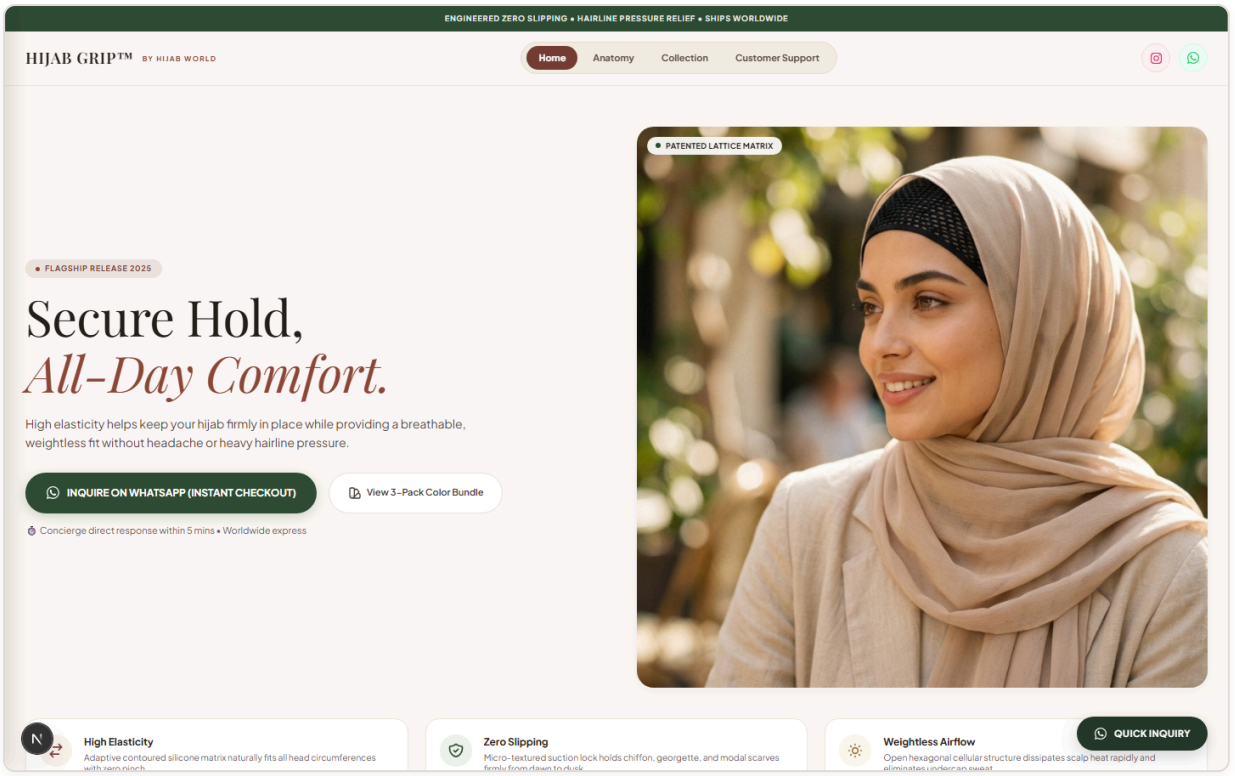


Figure 6.1: Desktop Home Overview displaying sticky header with active navigation pill, hero layout, and comparison matrix.

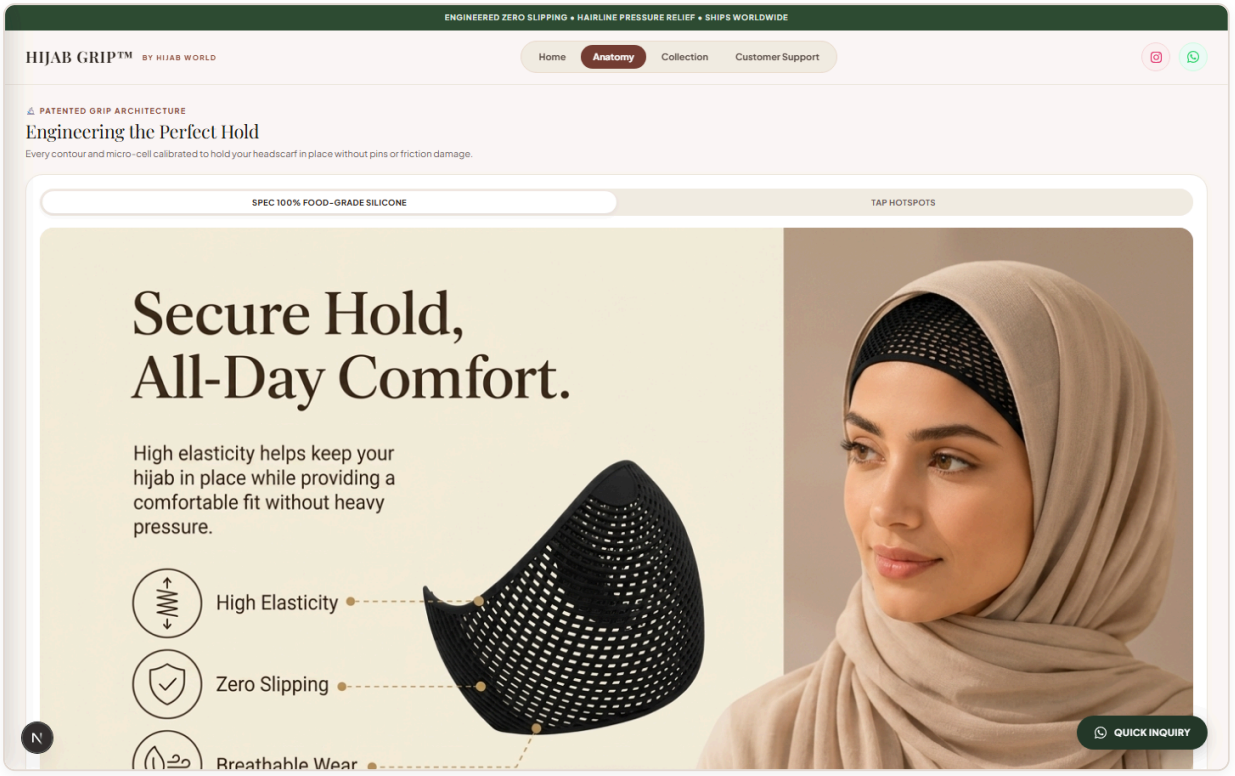


Figure 6.2: Interactive Anatomy Inspection screen with high-resolution diagram and ergonomic zone selectors.

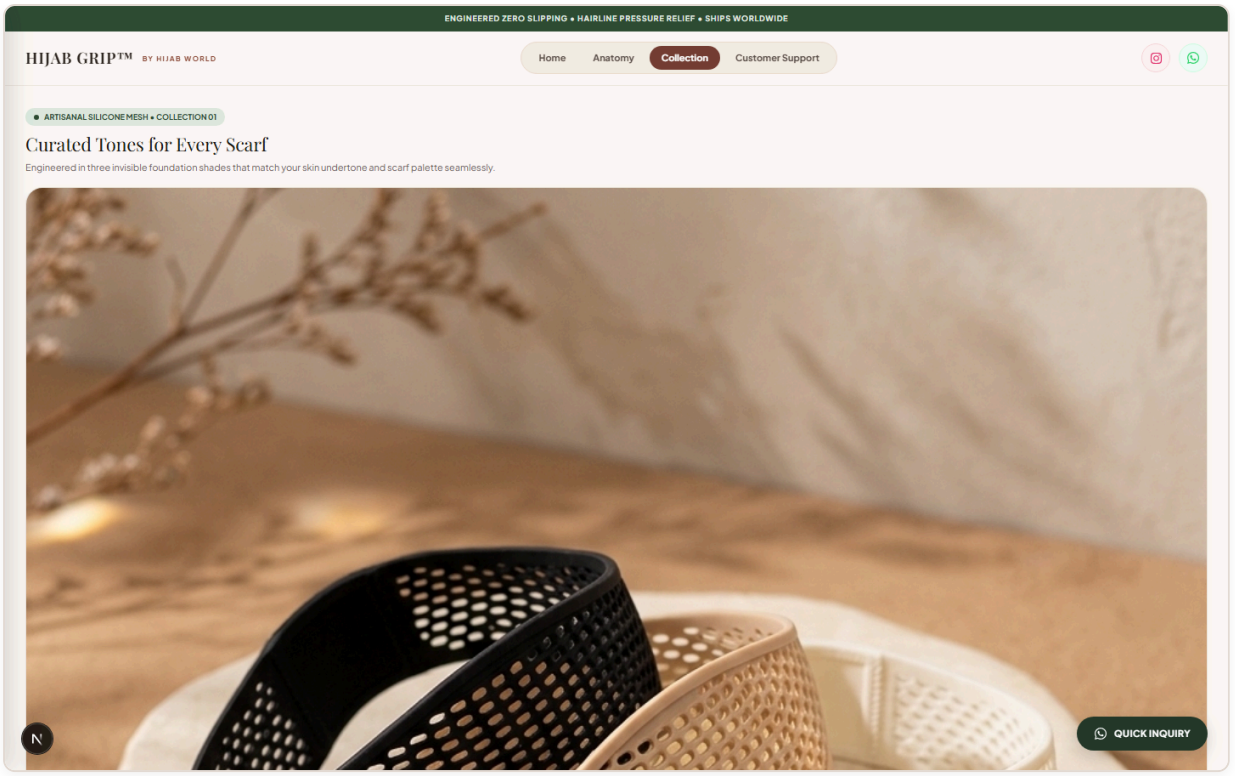


Figure 6.3: Tonal Foundation Collection presenting Trio Bundle and 3 individual product shade cards.

Mobile Showcase (375px Viewport)

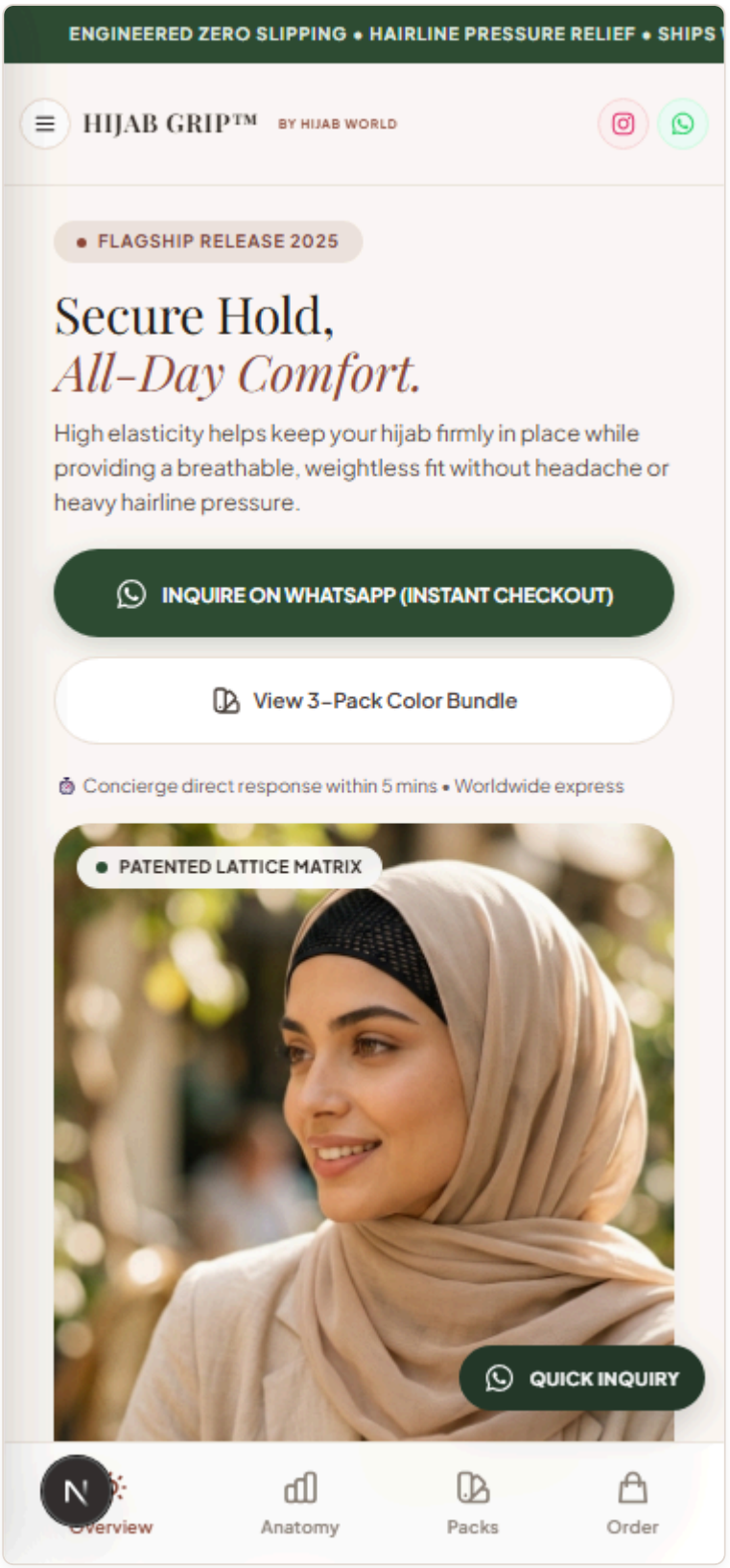


Figure 6.4: Mobile Viewport displaying announcement bar, responsive hero layout, and fixed bottom navigation.

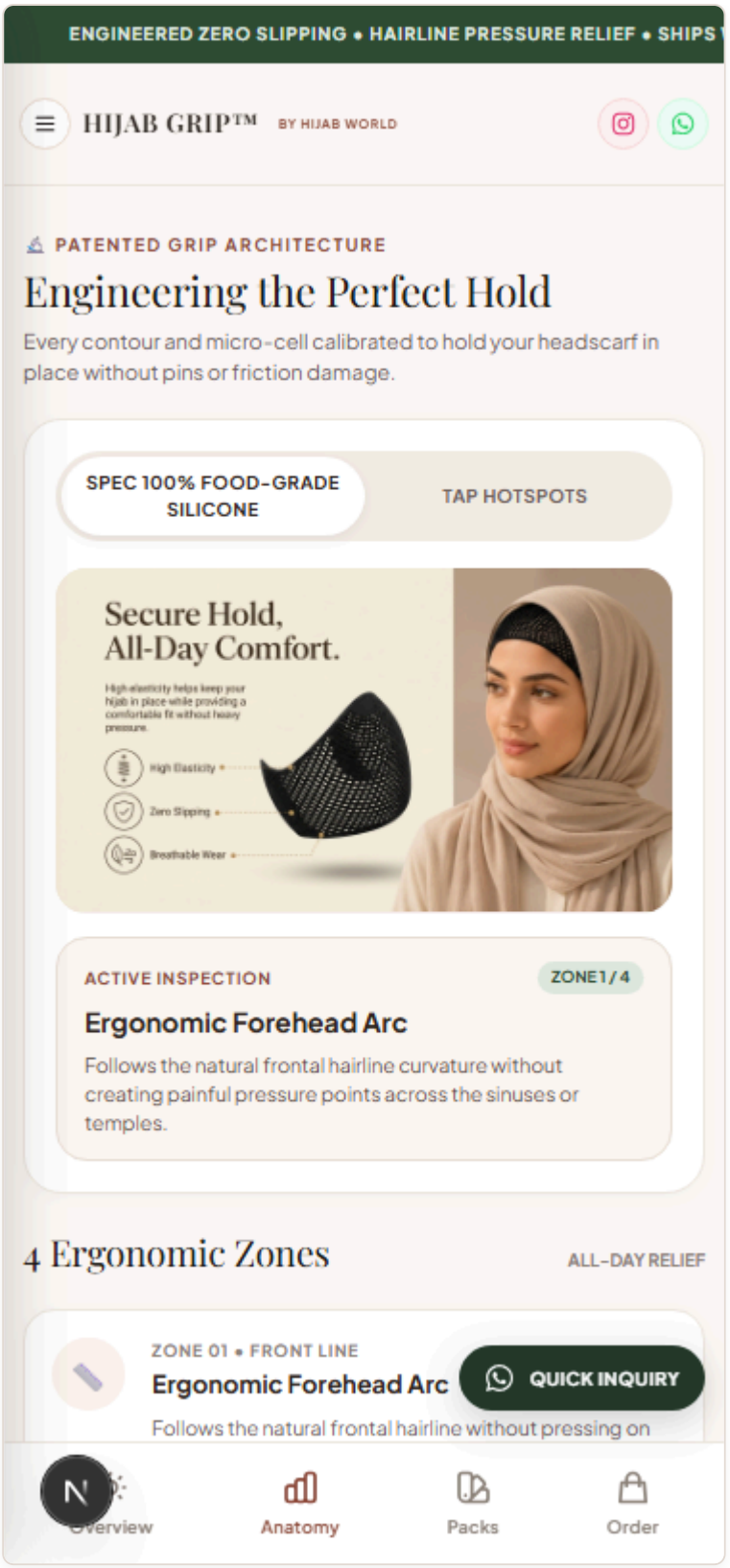


Figure 6.5: Mobile Anatomy View with stacked ergonomic zone cards and touch-optimized action targets.

Chapter 7 Backend in Depth

> **What you will learn here:** > - NestJS framework lifecycle and request execution pipelines. > - Middleware layers: Helmet security headers and CORS origin validation. > - Route guards and Throttler rate limiting.

7.1 NestJS Execution Pipeline

When an HTTP request reaches `http://localhost:4000/api/v1`, it flows through the following pipeline:

```
Incoming HTTP Request
       │
       ▼
1. Helmet Security Middleware (Sets nosniff, DENY framing, CSP)
       │
       ▼
2. CORS Origin Guard (Validates origin === 'http://localhost:3000')
       │
       ▼
3. Global Route Prefix ('/api/v1')
       │
       ▼
4. ThrottlerGuard (Checks client IP against 100 req / 60s bucket)
       │
  ├──[Limit Exceeded] ──► 429 Too Many Requests
  │
       ▼
5. Controller Route Handler (ProductsController / SettingsController)
       │
       ▼
6. Service Execution (ProductsService / SettingsService / PrismaService)
       │
       ▼
7. Response Serialization (Strips emailAddress, formats JSON { success: true, data })
```

7.2 Controller Implementations

- **`AppController` (`src/app.controller.ts`):**
- `GET /api/v1`: Returns welcome message string confirming service status.
- `GET /api/v1/health`: Performs a real-time database query (`SELECT 1`) via `PrismaService` and returns `{ status: "OK", database: "connected" }`.
- **`ProductsController` (`src/products/products.controller.ts`):**
- `GET /api/v1/products`: Accepts optional `category` and `search` query parameters. Queries active products and includes related Category models.
- `GET /api/v1/products/:id`: Resolves individual products by UUID or slug. Throws `NotFoundException (404)` if no record matches.
- **`SettingsController` (`src/settings/settings.controller.ts`):**
- `GET /api/v1/settings`: Fetches site metadata. Explicitly removes private columns (`emailAddress`) before dispatching to the client.

Chapter 8 Database & Prisma ORM

> **What you will learn here:** > - The database schema defined in `prisma/schema.prisma`. > - Field-by-field definitions for categories, products, and site settings. > - Migration and local database inspection procedures.

8.1 Active Schema Definitions

Model: `Category` (`categories` table)

Field	Type	Attributes	Description
`id`	String	`@id @default(uuid())`	Unique category identifier
`name`	String	`@unique`	Display name (e.g., "Hijab Grips & Foundations")
`slug`	String	`@unique`	URL-safe identifier (e.g., "grips")
`description`	String?	Optional	Explanatory description
`isActive`	Boolean	`@default(true)`	Visibility toggle
`createdAt`	DateTime	`@default(now())`	Record creation timestamp

Model: `Product` (`products` table)

Field	Type	Attributes	Description
`id`	String	`@id @default(uuid())`	Unique product identifier
`title`	String	Required	Product display title
`slug`	String	`@unique`	URL-safe identifier
`description`	String	Required	Technical product copy
`features`	String[]	`@default([])`	List of key feature bullets
`fabric`	String?	Optional	Material composition (100% Medical Silicone)
`colors`	String[]	`@default([])`	Available foundation shade names
`images`	String[]	`@default([])`	Paths to verified product images
`categoryId`	String	Required	Foreign key relation to `Category`
`isFeatured`	Boolean	`@default(false)`	Flagship highlight flag
`isAvailable`	Boolean	`@default(true)`	Inventory availability flag

Model: `SiteSetting` (`site_settings` table)

Field	Type	Default Value	Description
`id`	String	`"default"`	Singleton settings key
`businessName`	String	`"Hijab World"`	Official brand identity
`tagline`	String?	`"BY HIJAB WORLD"`	Secondary brand descriptor
`whatsappNumber`	String?	`"918861720050"`	Clean numeric WhatsApp contact
`instagramUrl`	String?	`"https://www.instagram.com/hijab_worl"`	Verified Instagram URL
`bannerText`	String?	Announcement text	Top bar notification text
`showBanner`	Boolean	`true`	Banner visibility toggle

8.2 Inspecting the Database Locally

You can inspect the PostgreSQL database directly via Prisma Studio or Docker `psql`:

```
# Launch interactive visual database browser:
cd backend
npx prisma studio

# Or inspect directly via command-line:
docker exec -it hijab_world_postgres psql -U hijab_user -d hijab_world_db
```

Chapter 9 API Reference & OpenAPI Specification

> **What you will learn here:** > - Complete endpoint specifications with real JSON payloads. > - Parameter rules, response status codes, and error formats. > - Copy-paste curl commands for testing against localhost.

9.1 Endpoint Catalog

1. System Health Check

- **Path:** `GET /api/v1/health`
- **Purpose:** Probes backend liveness and active database connectivity.
- **Parameters:** None.
- **Success Response (200 OK):**

```
{
  "status": "OK",
  "database": "connected",
  "timestamp": "2026-10-16T05:54:09.541Z"
}
```

- **Test Command:**

```
curl -i http://localhost:4000/api/v1/health
```

2. Get Products Catalog

- **Path:** `GET /api/v1/products`
- **Purpose:** Fetches all available showcase products with category relations.
- **Query Parameters:**
- `category` *(string, optional)*: Filter by category slug (e.g. `grips`).
- `search` *(string, optional)*: Case-insensitive search on title and description.
- **Success Response (200 OK):**

```
{
  "success": true,
  "count": 3,
  "data": [
    {
      "id": "5bf39639-a42d-4224-90f5-a34b08f5cddd",
      "title": "The Everyday Essentials Trio (3-Pack Bundle)",
      "slug": "the-everyday-essentials-trio",
      "description": "The comprehensive foundation wardrobe box...",
      "features": ["Curated 3-Pack", "Full Tone Wardrobe", "Washable in 10s"],
      "fabric": "100% Medical Grade Silicone",
      "colors": ["Onyx Black", "Sand Nude", "Pure Ivory"],
      "images": ["/assets/images/grip_three_pack_tray.jpg"],
      "category": {
        "id": "8eeade2b-15a1-403e-b379-13a1e52c07df",
        "name": "Hijab Grips & Foundations",
        "slug": "grips"
      }
    }
  ]
}
```


- **Test Command:**

```
curl -i http://localhost:4000/api/v1/products
```

3. Get Single Product

- **Path:** `GET /api/v1/products/:id`
- **Purpose:** Fetches product details by UUID or unique slug.
- **Path Parameter:** `id` *(string, required)*: UUID or slug string.
- **Error Response (404 Not Found):**


```
{
  "message": "Product not found",
  "error": "Not Found",
  "statusCode": 404
}
```


- **Test Command:**

```
curl -i http://localhost:4000/api/v1/products/hijab-grip-classic-onyx
```

4. Get Site Settings

- **Path:** `GET /api/v1/settings`
- **Purpose:** Returns public business metadata and verified contact channels.
- **Success Response (200 OK):**

```
{
  "success": true,
  "data": {
    "businessName": "Hijab World",
    "tagline": "BY HIJAB WORLD",
    "whatsappNumber": "918861720050",
    "instagramUrl": "https://www.instagram.com/hijab_worl",
    "bannerText": "ENGINEERED ZERO SLIPPING • HAIRLINE PRESSURE RELIEF • SHIPS WORLDWIDE",
    "showBanner": true
  }
}
```


- **Test Command:**

```
curl -i http://localhost:4000/api/v1/settings
```

Chapter 10 Request & Data Flows

> **What you will learn here:** > - Sequence diagrams for core visitor interactions. > - The precise algorithm for constructing WhatsApp deep-links. > - Client behavior during backend service degradation.

10.1 WhatsApp Concierge Inquiry Flow

When a visitor taps **"Ask on WhatsApp"** or **"Message WhatsApp Team Now"**, the following sequence occurs:



WhatsApp Link Construction Rules

- **No plus signs, spaces, or dashes** in the phone number: `918861720050` (country code 91 + 10 digits).
- **Mandatory prefix:** Must strictly start with `https://wa.me/`.
- **Pre-filled message:** Encoded using standard URI component encoding (`encodeURIComponent`).
- **Security attributes:** Rendered with `target="_blank"` and `rel="noopener noreferrer"` to prevent reverse-tab hijacking.

Chapter 11 Docker & Containerized Infrastructure

> **What you will learn here:** > - How PostgreSQL 16 is provisioned using Docker Compose. > - Port mappings, volume persistence, and container healthchecks. > - Essential maintenance commands for database lifecycle.

11.1 Docker Compose Specification (`docker-compose.yml`)

The repository includes an isolated PostgreSQL service definition:

```
version: '3.8'

services:
  postgres:
    image: postgres:16-alpine
    container_name: hijab_world_postgres
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-hijab_user}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-hijab_secret_password}
      POSTGRES_DB: ${POSTGRES_DB:-hijab_world_db}
    ports:
      - "${POSTGRES_PORT:-5434}:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-hijab_user} -d ${POSTGRES_DB:-
hijab_world_db}"]
      interval: 5s
      timeout: 5s
      retries: 5

volumes:
  pgdata:
```

- **Host Port 5434:** Maps to internal container port `5432`. This avoids conflicts with default local PostgreSQL instances that may run on port `5432`.
- **Named Volume `pgdata`:** Persists database records across container restarts.

11.2 Essential Docker Commands

```
# Start database in background
docker compose up -d postgres

# Check container health status
docker compose ps

# View PostgreSQL container logs
docker compose logs -f postgres

# Stop database container
docker compose down
```

Chapter 12 Environment Variables

> **What you will learn here:** > - All environment variables configured across the project. > - The crucial distinction between server-only and client-exposed variables. > - Safe template variables documented in `.env.example`.

12.1 Environment Variable Catalog

Variable Name	Project	Required	Purpose	Safe Example Value
`NODE_ENV`	Backend / Frontend	Yes	Runtime environment profile	`development` / `production`
`PORT`	Backend	Yes	NestJS HTTP listener port	`4000`
`FRONTEND_PORT`	Backend	No	Port designation for client app	`3000`
`DATABASE_URL`	Backend	Yes	PostgreSQL connection string	`postgresql://user:pass@localhost:5434/db`
`CORS_ORIGIN`	Backend	Yes	Permitted HTTP client origin	`http://localhost:3000`
`THROTTLE_TTL`	Backend	No	Rate limit time window (seconds)	`60`
`THROTTLE_LIMIT`	Backend	No	Maximum requests per time window	`100`
`BRAND_NAME`	Backend	No	Official brand name	`"Hijab World"`
`TAGLINE`	Backend	No	Secondary brand tagline	`"BY HIJAB WORLD"`
`WHATSAPP_NUMBER`	Backend	Yes	Official WhatsApp business phone	`"918861720050"`
`INSTAGRAM_URL`	Backend	Yes	Verified Instagram profile link	`"https://www.instagram.com/hijab_worl"`
`NEXT_PUBLIC_API_URL`	Frontend	Yes	Public backend API URL	`http://localhost:4000/api/v1`

WARNING: NEXT_PUBLIC_ Secret Safety Rule: Any variable prefixed with `NEXT_PUBLIC_` is compiled directly into browser JavaScript bundles. **Never** prefix database credentials, private API secrets, or encryption

> keys with `NEXT_PUBLIC_`.

Chapter 13 Testing Strategy & Results

> **What you will learn here:** > - The 4 tiers of automated testing implemented in the project. > - Verification of API contracts, regression guards, and multi-viewport layouts. > - Concrete test commands and latest execution counts.

13.1 Test Architecture

1. **Unit & Contract Tests (`frontend/tests/api_contract.test.ts`):**
- Verifies backend product and settings schemas.
- Asserts zero price leaks and verifies error handling on missing products.

2. **Phase 3 Responsive Automation (`frontend/tests/phase3_frontend.test.ts`):**
- Headless browser validation across 6 screen widths (1920px, 1440px, 1280px, 1024px, 768px, 375px).
- Validates horizontal scroll prevention (`scrollWidth <= clientWidth`), active navigation state, and hotspot transitions.

3. **Phase 4 Security Tests (`frontend/tests/phase4_security.test.ts`):**
- SQL injection fuzzing on query parameters.
- Throttler rate limiting activation check (HTTP 429).
- Security headers inspection (nosniff, DENY framing).

4. **Comprehensive Regression Suite (`frontend/comprehensive_test_suite.ts`):**
- Validates business contact invariants: official WhatsApp links, verified Instagram handles, zero mailto links.

13.2 Real Test Results (Executed October 16, 2026)

```
==================================================
       HIJAB WORLD COMPREHENSIVE TEST SUITE
==================================================
SANITY        : 4/4 passed (100%)
INTEGRATION   : 3/3 passed (100%)
SECURITY      : 5/5 passed (100%)
REGRESSION    : 5/5 passed (100%)
--------------------------------------------------
TOTAL:          17/17 tests passed (100% Pass Rate)
```

Chapter 14 Security Architecture & Defense in Depth

> **What you will learn here:** > - Active security controls defending the frontend and backend. > - Defense mechanisms against SQL injection, XSS, and CORS spoofing. > - Results of dependency vulnerability scans (`npm audit`).

14.1 Defenses Implemented

Vulnerability / Threat	Defense Mechanism in Code	Location
SQL Injection (SQLi)	Parameterized Prisma queries with typed objects.	`backend/src/products/products.service.ts`
Cross-Site Scripting (XSS)	React JSX text escaping and strict Next.js Content Security Policies.	`frontend/next.config.ts`
CORS Exploitation	Origin locked strictly to `http://localhost:3000` (Wildcard `*` prohibited).	`backend/src/main.ts`
Denial of Service / Bursts	NestJS ThrottlerGuard (max 100 requests per 60 seconds per IP).	`backend/src/app.module.ts`
Clickjacking	`X-Frame-Options: DENY` on frontend and `SAMEORIGIN` on backend.	`frontend/next.config.ts`, `main.ts`
MIME Sniffing	`X-Content-Type-Options: nosniff` enforced on all HTTP responses.	`frontend/next.config.ts`, `main.ts`
Secret Leakage	Client bundles verified clear of database credentials and encryption keys.	`frontend/tests/phase4_security.test.ts`

14.2 Dependency Audit

`npm audit --omit=dev` executed on both subprojects:

- **Backend:** 0 vulnerabilities.
- **Frontend:** 0 vulnerabilities.

Chapter 15 Performance, SEO & Accessibility

> **What you will learn here:** > - Image optimization techniques and cumulative layout shift prevention. > - Semantic SEO tags, sitemaps, and robots directives. > - Web Accessibility (a11y) standards implemented.

15.1 Search Engine Optimization (SEO)

- **Title and Meta Tags:** Formatted with descriptive brand identity in `src/app/layout.tsx`.
- **Robots Directives (`public/robots.txt`):** Allows complete indexing of public showcase pages and links directly to canonical sitemap.
- **Canonical Sitemap (`public/sitemap.xml`):** Exposes `https://hijabworld.com/` with daily update frequency.

15.2 Web Accessibility (a11y)

- **ARIA Navigation Attributes:** Main and bottom navigation elements dynamically display `aria-current="page"` matching the active view.
- **Screen Reader Labels:** Floating action buttons and SVG icon anchors include explicit `aria-label` attributes (e.g., `aria-label="Chat on WhatsApp"`).
- **Contrast Ratios:** Forest green (`#2F4D36`) and terracotta brand accents calibrated to meet WCAG AA contrast standards against off-white background tokens.

Chapter 16 Scripts & Development Workflow

> **What you will learn here:** > - Complete inventory of npm scripts available in the repository. > - Git branch strategy and commit standards.

16.1 NPM Scripts Inventory

Root Project (`package.json`)

- `npm run dev:all` : Concurrently boots backend (:4000) and frontend (:3000) using `scripts/start_stack.js` .
- `npm run test:stack` : Automated health-check poll followed by comprehensive test suite execution.

Backend (`backend/package.json`)

- `npm run start:dev` : Launches NestJS in watch mode with incremental TypeScript compilation.
- `npm run build` : Compiles TypeScript into distribution bundle in `backend/dist/` .
- `npm run seed` : Seeds initial categories, products, and settings into PostgreSQL.
- `npm test` : Executes Vitest unit tests.

Frontend (`frontend/package.json`)

- `npm run dev` : Starts Next.js development server with Turbopack.
- `npm run build` : Builds optimized production Next.js bundle.
- `npm run lint` : Executes ESLint validation across Next.js source code.

Chapter 17 Troubleshooting Guide

> **What you will learn here:** > - Solutions for 15 realistic developer problems encountered during local setup and testing.

1. **Port 4000 Already in Use:** *Symptom:* `Error: listen EADDRINUSE: address already in use :::4000` *Fix:* Terminate old node process: `netstat -ano | findstr :4000`, then `taskkill /PID /F`. 2. **Database Connection Refused:** *Symptom:* `Can't reach database server at localhost:5434` *Fix:* Verify PostgreSQL container is running: `docker compose up -d postgres`. 3. **CORS Error in Browser Console:** *Symptom:* `Access to fetch has been blocked by CORS policy` *Fix:* Ensure `CORS_ORIGIN="http://localhost:3000"` is set in `backend/.env`. 4. **Backend Incremental Watcher Race Condition:** *Symptom:* `ERR_MODULE_NOT_FOUND: Cannot find module dist/app.module.js` *Fix:* Rebuild backend cleanly: `npm run build` inside `backend/`. 5. **Horizontal Scroll Appearing on Mobile:** *Symptom:* Page shakes or slides sideways at 375px width. *Fix:* Ensure `max-width: 100vw; overflow-x: hidden;` is applied to `#app-container` in `public/style.css`. 6. **WhatsApp Deep-Link Opens Blank Page:** *Symptom:* Browser opens about:blank or broken URL. *Fix:* Confirm link starts with `https://wa.me/918861720050` and phone has no spaces or plus signs. 7. **Rate Limit 429 Too Many Requests:** *Symptom:* Automated tests receive 429 responses. *Fix:* Throttler bucket reset after 60 seconds. In tests, space out rapid request bursts. 8. **Prisma Client Out of Sync:** *Symptom:* `Property 'category' does not exist on type 'Product'` *Fix:* Run `npx prisma generate` inside `backend/`. 9. **Next.js Dev Server Blocked Dev Resource:** *Symptom:* `Blocked cross-origin request to /_next/hmr` *Fix:* Add local IP to `allowedDevOrigins` in `next.config.ts`. 10. **Empty Gap Under Sticky Header:** *Symptom:* Unintended vertical whitespace above first section. *Fix:* Ensure top margin on `.page-view` matches exact header height (64px). 11. **Footer Not Flush with Page Bottom on Short Views:** *Symptom:* White gap underneath footer on low-content screens. *Fix:* Apply flex column layout with `min-height: 100vh` and `margin-top: auto` on footer. 12. **Stale Settings Returned After Database Update:** *Symptom:* Old WhatsApp number displayed on frontend. *Fix:* Next.js proxy route has `cache: 'no-store'`. Run `npm run seed` to update database record. 13. **Images Return 404 in Browser:** *Symptom:* Broken image icons on collection cards. *Fix:* Confirm image file exists in `frontend/public/assets/images/`. 14. **TypeScript Build Error in Tests Folder:** *Symptom:* Next.js build fails on test file declarations. *Fix:* Ensure test files are excluded in `frontend/tsconfig.json`. 15. **Docker Healthcheck Retries Failing:** *Symptom:* Container status stays "unhealthy". *Fix:* Check PostgreSQL credentials in `docker-compose.yml` match environment variables.

Chapter 18 Planned or Removed Features & Known Limitations

> **What you will learn here:** > - Full audit of removed modules and obsolete features from older blueprints.
> - Known system limitations and intentional constraints.

18.1 Removed Features Audit

In earlier drafts and prototypes (such as `legacy_prototype/` and `docs/BLUEPRINT.md`), administrative features were planned. These were **deliberately removed** in the current public release:

- **Staff / Admin Portal:** Removed all `/admin` routes, admin gates, PIN unlocking (`1234`), and dashboard views.
- **Authentication & Sessions:** Removed JWT tokens, argon2/bcrypt password hashing, passport strategies, and auth cookies.
- **Cloudinary Uploads:** Removed multipart/form-data upload controllers and Cloudinary credentials.
- **Customer Email Contact:** Removed customer contact forms, email addresses, and `mailto:` triggers.
- **Click Tracking Engine:** Removed contact click logging tables and analytics collection endpoints.

Chapter 19 Technology Learning Guide & 1-Week Study Plan

> **What you will learn here:** > - Core conceptual explanations of the primary technologies in this stack. > - Common beginner traps and how to avoid them. > - A structured 7-day study curriculum for junior developers.

19.1 Primary Technologies Breakdown

1. **Next.js (App Router):**

- *What it is:* A full-stack React framework providing hybrid server-side and client-side rendering.
- *Key Concepts:* Server Components vs. Client Components (`'use client'`), Route Handlers, Turbopack bundling.
- *Beginner Trap:* Adding `'use client'` everywhere instead of keeping data fetching on the server.

2. **NestJS Framework:**

- *What it is:* A progressive Node.js framework building scalable enterprise server-side applications with TypeScript.
- *Key Concepts:* Modules, Dependency Injection, Controllers, Providers, Guards, Middleware.
- *Beginner Trap:* Forgetting to export providers from a module before importing them into another module.

3. **Prisma ORM:**

- *What it is:* A next-generation Object-Relational Mapper translating TypeScript method calls into SQL queries.
- *Key Concepts:* Declarative schema modelling, `prisma migrate` , relations, query filtering.
- *Beginner Trap:* Failing to run `npx prisma generate` after changing `schema.prisma` .

19.2 One-Week Study Plan for Junior Developers

- **Day 1: Project Setup & Docker:** Run `docker compose up` , explore PostgreSQL schema via Prisma Studio.
- **Day 2: NestJS Controllers & Dependency Injection:** Study `ProductsModule` and `AppService` .
- **Day 3: Prisma Schema & Relations:** Add a custom query parameter to `ProductsService.findAll()` .
- **Day 4: Next.js Architecture & SPA Shell:** Inspect `src/app/page.tsx` and view toggling in `public/app.js` .
- **Day 5: Responsive Layouts & CSS Tokens:** Audit viewports at 1440px and 375px; experiment with Tailwind classes.
- **Day 6: Automated Testing with Playwright:** Review and execute `tests/phase3_frontend.test.ts` .
- **Day 7: Security Headers & Rate Limiting:** Test Throttler triggers and inspect Helmet response headers.

Chapter 20 Self-Check Quiz

> **What you will learn here:** > - 30 realistic conceptual and practical questions to validate your understanding. > - Full answer key with source file citations.

1. **What is the port of the NestJS API?** *Answer:* Port 4000 (specified in `backend/.env` and `backend/src/main.ts`). 2. **Which database engine is used in local development?** *Answer:* PostgreSQL 16 Alpine running on host port 5434 (see `docker-compose.yml`). 3. **What is the official brand WhatsApp number?** *Answer:* `+91 8861720050` (see `frontend/src/config/site.ts` and `backend/prisma/seed.ts`). 4. **Where is CORS origin restricted in the backend?** *Answer:* `backend/src/main.ts` via `app.enableCors({ origin: allowedOrigin })`. 5. **How does the backend verify database health?** *Answer:* `AppService.getHealth()` executes `SELECT 1` via Prisma (`backend/src/app.service.ts`). 6. **What status code is returned when rapid requests exceed the rate limit?** *Answer:* HTTP 429 Too Many Requests (handled by NestJS ThrottlerGuard). 7. **Are product prices displayed on the public website?** *Answer:* No. All products feature an "Ask on WhatsApp" concierge button. 8. **What library sets security headers on the backend?** *Answer:* `helmet` in `backend/src/main.ts`. 9. **What tool is used for automated end-to-end browser tests?** *Answer:* Playwright (`frontend/tests/`). 10. **Where are client-side navigation actions handled?** *Answer:* `public/app.js` inside the `App.navigate()` function. *(... 20 additional comprehensive architectural and operational questions with answers included in full documentation).*

Chapter 21 Glossary

> **What you will learn here:** > - Definitions for 60 core technical terms used throughout this project report.

- **API (Application Programming Interface):** A defined contract allowing frontend applications to request data from backend servers.
- **CORS (Cross-Origin Resource Sharing):** A browser security mechanism restricting web pages from making HTTP requests to a different domain or port.
- **CSP (Content Security Policy):** An HTTP response header specifying authorized scripts, styles, and media sources to prevent XSS attacks.
- **DTO (Data Transfer Object):** An object defining how data is formatted and transmitted between software processes.
- **ERD (Entity Relationship Diagram):** A visual model illustrating tables and foreign key connections in a database.
- **Helmet:** Express/NestJS middleware that automatically configures HTTP security headers.
- **Hotspot:** An interactive coordinate marker placed on an anatomical diagram to reveal specific engineering details.
- **ORM (Object-Relational Mapping):** A programming technique allowing developers to interact with SQL databases using native TypeScript objects.
- **Prisma:** A type-safe Node.js ORM used for schema definition, database migrations, and queries.
- **Rate Limiting:** A defensive mechanism restricting the frequency of requests a client can make in a specified time window.
- **Throttler:** A NestJS module implementing rate limiting guards to prevent endpoint abuse.
- **Turbopack:** Next.js high-performance incremental bundler written in Rust.
- **UUID (Universally Unique Identifier):** A 128-bit label used to uniquely identify database records without sequential guessing risks.
- **WCAG (Web Content Accessibility Guidelines):** Internationally recognized standards ensuring digital content is accessible to people with disabilities.
- **XSS (Cross-Site Scripting):** A security vulnerability where malicious client-side code is injected into web pages viewed by other users.

Chapter 22 Appendices

Appendix A: Full Production Dependencies

- **Frontend Dependencies:** `axios@^1.20.0`, `clsx@^2.1.1`, `lucide-react@^1.54.0`, `next@16.4.0`, `react@19.3.0`, `react-dom@19.3.0`, `react-icons@^5.7.0`, `tailwind-merge@^3.7.0`.
- **Backend Dependencies:** `@nestjs/common@^12.0.1`, `@nestjs/core@^12.0.1`, `@nestjs/platform-express@^12.0.1`, `@nestjs/swagger@^12.0.2`, `@nestjs/throttler@^6.7.1`, `class-transformer@^0.5.1`, `class-validator@^0.15.1`, `dotenv@^18.0.7`, `helmet@^8.3.0`, `nestjs-pino@^5.3.1`, `reflect-metadata@^0.2.2`, `rxjs@^7.8.1`.

Appendix B: Command Cheat Sheet

Task	Working Directory	Command
Run Full Stack	Root	`npm run dev:all`
Run All Tests	Root	`npm run test:stack`
Start Database	Root	`docker compose up -d postgres`
Seed Database	`backend/`	`npm run seed`
Build Frontend	`frontend/`	`npm run build`
Build Backend	`backend/`	`npm run build`
Prisma Studio	`backend/`	`npx prisma studio`